Earn To Die

Technical Documentation & Developer Reference

Shehzil Umerani · Nameer Subhani · March 2026

Language	C++17
Framework	Raylib
Files	`Car.hpp`, `Terrain.hpp`, `Main.cpp`
Version	0.1.0 (Initial Prototype)
Status	In Development

This document describes the current architecture and is a living reference for future development.

Contents

1 Project Overview **2**

2 Architecture **2**
2.1 Class Hierarchy . . . 2

3 Car System (`Car.hpp`) **2**
3.1 Member Variables . . . 3
3.2 Public Interface . . . 3
3.2.1 Constructor . . . 3
3.2.2 Movement: `updateX(float dt)` . . . 3
3.2.3 Movement: `updateY(float dt)` . . . 3
3.2.4 Jumping: `jump()` . . . 4
3.2.5 Terrain Effect: `applyTerrainEffect(double modifier)` . . . 4
3.3 Subclasses . . . 4

4 Terrain System (`Terrain.hpp`) **4**
4.1 Interface . . . 4
4.2 Terrain Types . . . 4
4.3 TerrainManager . . . 4

5 Game Loop (`Main.cpp`) **5**
5.1 Initialization . . . 5
5.2 Input Handling . . . 5
5.3 Parallax Background . . . 5
5.4 Camera System . . . 5

6 Under Development **5**
6.1 Zombie System (`Zombie.hpp`) . . . 5
6.2 Extended Terrain Types (`Terrain.hpp`) . . . 6
6.3 New Background Packs . . . 6
6.4 New Vehicles (`Car.hpp`) . . . 6
6.5 Sprites & Animations (`Car.hpp`) . . . 7
6.6 Upgrade System — Tyres & Gun (`Car.hpp`, `Gun.hpp`) . . . 7
6.6.1 Tyre Upgrades . . . 7
6.6.2 Gun Upgrades . . . 7
6.7 UI Screens (`UI.hpp`) . . . 8
6.8 Summary of Planned File Changes . . . 8

7 Test Cases **8**
7.1 Car Physics Tests . . . 9
7.2 Terrain Tests . . . 10

8 Build & Dependencies **10**

9 Revision History **11**

1 Project Overview

This project is a 2D side-scrolling car game built in C++ using the **Raylib** graphics library. The player controls a vehicle that moves horizontally across a procedurally assembled terrain map, with physics-based movement, jumping, friction, gravity, and a parallax scrolling background system.

File	Responsibility
`Car.hpp`	Vehicle class hierarchy, physics, rendering
`Terrain.hpp`	Terrain types and terrain map manager
`Main.cpp`	Game loop, input handling, camera, background layers

2 Architecture

2.1 Class Hierarchy

`Class`	Description
`Car`	Base class; physics, state, drawing
`SportsCar`	Fast, low-traction subclass
`Truck`	Slow, high-traction subclass
`Buggy`	Balanced, mid-weight subclass
`Terrain`	Pure virtual interface
`Grass`	Speed modifier 1.0
`Sand`	Speed modifier 0.6
`Swamp`	Speed modifier 0.4
`TerrainManager`	Generates and queries the map

3 Car System (`Car.hpp`)

3.1 Member Variables

Variable	Type	Description
`name`	`string`	Human-readable vehicle name
`speed`	`double`	Current horizontal speed
`upspeed`	`double`	Vertical velocity during jump
`jumpTime`	`double`	Reserved for airborne duration tracking
`Xacceleration`	`double`	Horizontal input (-2, 0, or $+2$)
`Yacceleration`	`int`	Gravity constant (10 units/s^2)
`x`, `y`	`double`	World-space position
`mass`	`int`	Car mass (reserved)
`tireTraction`	`double`	Acceleration multiplier
`maxSpeed`	`int`	Hard speed cap
`maxAcceleration`	`int`	Acceleration cap (reserved)
`gunLevel`	`static int`	Shared weapon upgrade level
`tireLevel`	`static int`	Shared tyre upgrade level
`jumpspeed`	`int`	Initial jump velocity (5 units)
`Grounded`	`bool`	True when car is on the ground

3.2 Public Interface

3.2.1 Constructor

```
Car(string n, int mS, int mA, double tT, int m)
```

Initialises all state. Cars start at `(0,0)`, at rest, and grounded.

3.2.2 Movement: `updateX(float dt)`

1. If no input, apply friction of 4.0 units/s.
2. Otherwise: `speed += Xacceleration * tireTraction * dt`.
3. Clamp to `[-maxSpeed, +maxSpeed]`.
4. Advance: `x += speed * dt * 300`.

> **Note:** The factor of `300` converts normalised speed into screen pixels. Adjust to tune game feel.

3.2.3 Movement: `updateY(float dt)`

1. Only runs when `Grounded == false`.
2. `upspeed -= Yacceleration * dt` (gravity).
3. `y -= upspeed * dt * 100`.
4. If `y >= 300`: land, reset `upspeed`, set `Grounded = true`.

> **Hardcoded ground:** Ground is fixed at `y = 300`. Must be made dynamic for terrain height variation.

3.2.4 Jumping: `jump()`

```
void jump() {
    if (Grounded) { upspeed = jumpspeed; Grounded = false; }
}
```

3.2.5 Terrain Effect: `applyTerrainEffect(double modifier)`

Clamps speed to a ±1.0 floor on terrain transitions. The modifier is accepted but currently unused.

> **Known issue:** Terrain modifier is not applied to speed. Terrain has no gameplay effect yet.

3.3 Subclasses

Class	maxSpeed	maxAccel	tireTraction	mass
SportsCar	80	15	0.60	10
Truck	30	8	1.20	40
Buggy	50	10	0.75	5

4 Terrain System (`Terrain.hpp`)

4.1 Interface

```
class Terrain {
  public:
    virtual double getSpeedModifier() = 0;
    virtual char   getSymbol()        = 0;
};
```

4.2 Terrain Types

Class	Speed Modifier	Symbol
Grass	1.0	.
Sand	0.6	~
Swamp	0.4	S

4.3 TerrainManager

```
class TerrainManager {
    vector<Terrain*> map;
  public:
    void     generateMap();          // 30 Grass + 30 Sand + 30 Swamp
    Terrain* getTerrain(int index); // index % map.size() for looping
    void     draw(float offset);     // debug draw
};
```

5 Game Loop (`Main.cpp`)

5.1 Initialization

```
InitWindow(1400, 600, "Test Car + Terrain");
Camera2D cam = { 0 };
cam.offset   = (Vector2){ 400, -100 };
cam.zoom     = 1.0f;
```

5.2 Input Handling

Key	Action
D	Accelerate right
A	Accelerate left
W	Jump
S	Brake
Q	Quit
1-4	Switch background pack

5.3 Parallax Background

```
float speeds[LAYERS] = {0.02f, 0.05f, 0.1f, 0.2f, 0.4f, 0.8f};
offsets[i] -= car.getSpeed() * dt * 200 * speeds[i];
```

Each layer is drawn twice to create seamless infinite scrolling.

5.4 Camera System

```
float leadOffset = speedRatio * 100.0f;
cam.target.x    += (desiredX - cam.target.x) * 2.0f * dt;
```

Clamped to within ±200 px of the car.

6 Under Development

This section describes all features currently planned or in progress. Each subsection explains what is being built, why it is needed, and how it fits into the existing codebase. The full code for these features lives in the separate `.hpp` source files.

6.1 Zombie System (`Zombie.hpp`)

Zombies are the primary obstacle in the game. They spawn ahead of the car at timed intervals, move toward the camera on their own, and are removed when killed by the car or by bullets. A single abstract `Zombie` base class defines the shared interface — position, health, score value, and a `scroll()` method that keeps zombies locked to world space as the camera moves. Four concrete types are implemented, each as a subclass:

Type	HP	Score	Behaviour
`WalkerZombie`	1	10	Slow, upright walk. Most common spawn.
`RunnerZombie`	2	25	Fast movement, harder to shoot before reaching the car.
`TankZombie`	10	100	Very slow but takes many hits. Displays a health bar above it.
`CrawlerZombie`	1	15	Ground-level, low profile makes it harder to hit with bullets.

All zombies are drawn using Raylib primitive shapes (rectangles and circles) until sprite assets are ready. Spawning is handled in `Main.cpp` via a timer that decreases in interval over time, ramping up difficulty the further the player travels.

6.2 Extended Terrain Types (`Terrain.hpp`)

Three new terrain types are being added alongside the existing Grass, Sand, and Swamp. Each inherits from the same `Terrain` base class and overrides the speed modifier to produce a distinct driving feel. A new `getColor()` virtual method is also being added to the base class so tiles can eventually be rendered as coloured ground strips rather than debug text characters.

Type	Modifier	Symbol	Effect
`Ice`	1.4	`I`	Slippery surface. Car moves faster but is harder to slow down.
`Mud`	0.25	`M`	Heavy drag. The car nearly stops without sustained throttle.
`Rock`	0.8	`R`	Slightly rough. Minor speed reduction, good for jump ramps.

These types are appended to `TerrainManager::generateMap()` so they appear after the original biome sequence, creating a progression from easy to difficult terrain as the player drives further.

6.3 New Background Packs

The existing system supports four parallax background packs loaded from `assets/1/` through `assets/4/`. Two additional packs are being added — a night city environment (`assets/5/`) and a snow biome (`assets/6/`). Each pack follows the same folder structure: one `background.png` base layer plus five foreground layers named `1.png` through `5.png`.

The active pack currently switches manually via the number keys. The plan is to auto-switch packs based on distance travelled so the environment changes naturally as the player progresses through the level.

6.4 New Vehicles (`Car.hpp`)

Two new vehicle subclasses are being added to expand player choice. Both inherit from `Car` and only differ in their constructor parameters — no new logic is required.

Class	maxSpeed	maxAccel	Traction	Mass	Character
`MonsterTruck`	40	12	1.50	60	Tank-like. Crushes zombies easily but slow on open road.
`Hatchback`	65	11	0.70	15	All-rounder. Between SportsCar and Buggy in every stat.

The vehicle selection screen (part of the UI system below) will allow the player to choose between all five vehicle types before starting a run.

6.5 Sprites & Animations (`Car.hpp`)

A `SpriteAnim` struct is being added to `Car.hpp` to handle frame-by-frame sprite sheet animation. It stores a `Texture2D` sheet, a frame width, a frame count, and a timer that advances the current frame at a configurable rate. Each `Car` instance will hold a `wheelAnim` member of this type. The frame rate of the animation is tied to the car's current speed — wheels spin faster at high speed and slow down when braking, making the animation feel physically grounded.

This same struct will also be used to animate zombie walk cycles once sprite sheets are created for them.

6.6 Upgrade System — Tyres & Gun (`Car.hpp`, `Gun.hpp`)

6.6.1 Tyre Upgrades

The `tireLevel` static variable already exists in `Car` but is currently unused. The upgrade system wires it into the physics in `updateX`: each tyre level increases traction by 15% and reduces the friction coefficient by 0.4, down to a minimum of 1.0. This means higher-level tyres accelerate more aggressively and grip better on difficult terrain like Mud and Swamp, making the upgrade feel meaningful in gameplay.

6.6.2 Gun Upgrades

A new `Gun` class (in `Gun.hpp`) manages a list of active bullets and a cooldown timer. The player fires by holding the Space key. Gun level controls two things: the cooldown between shots, and the damage each bullet deals. Levels 1 and 2 fire single bullets; level 3 and above fire two bullets simultaneously. At level 5 each bullet deals 3 damage, allowing even Tank zombies to be cleared quickly.

Level	Cooldown (s)	Damage	Bullets per shot
1	0.50	1	1
2	0.35	1	1
3	0.20	2	2
4	0.15	2	2
5	0.10	3	2

Both upgrade levels are shared across all car instances as static members, so upgrades persist across vehicle swaps.

6.7 UI Screens (`UI.hpp`)

All UI screens are managed through a `GameState` enum that controls which screen the game renders each frame. The main loop in `Main.cpp` switches on this value inside the `BeginDrawing()` block. Five states are defined:

State	Description
`START`	Title screen with Play, Upgrades, and Quit buttons. Shows control hints.
`PLAYING`	Active gameplay. Renders the game world plus a HUD showing score, distance, and current upgrade levels.
`PAUSED`	Drawn as a semi-transparent overlay over the frozen game world. Offers Resume, Upgrades, and Main Menu.
`UPGRADE`	Full-screen upgrade shop. Shows pip indicators for tyre and gun level, escalating costs per level, and current money.
`GAME_OVER`	Displays final score and distance. Offers Retry, Upgrades, and Main Menu.

A shared `DrawButton()` helper handles hover detection, colour feedback, and click events for all buttons across all screens. Pressing `Escape` toggles between `PLAYING` and `PAUSED`. Money is earned by killing zombies and is spent in the upgrade screen; it persists between runs within a session.

6.8 Summary of Planned File Changes

File	Action	What changes
`Car.hpp`	Edit	Add `SpriteAnim` struct and `wheelAnim` member; wire `tireLevel` into friction and traction; add `MonsterTruck` and `Hatchback` subclasses; expose static upgrade levels via reference getters
`Terrain.hpp`	Edit	Add `Ice`, `Mud`, `Rock` classes; add `getColor()` to base interface; extend `generateMap()`
`Zombie.hpp`	New	Abstract `Zombie` base with `Walker`, `Runner`, `Tank`, `Crawler` subclasses
`Gun.hpp`	New	`Bullet` struct; `Gun` class with level-based fire rate and damage
`UI.hpp`	New	`GameState` enum; all five screen draw functions; shared `DrawButton` helper; HUD
`Main.cpp`	Edit	Include new headers; add `GameState` switch; zombie spawn and collision loop; bullet–zombie collision; score and money tracking; auto background pack switching; proper memory cleanup on exit

7 Test Cases

Each test case below is written with a setup, the action performed, the expected result, and the pass condition. Tests are grouped by system. A test passes when the actual output matches the expected output exactly.

7.1 Car Physics Tests

TC-C01 — Initial State

Setup: Construct a `Buggy` with `tireLevel = 1`. *Action:* Read `GetX()`, `GetY()`, `getSpeed()` immediately after construction. *Expected:* All three return `0.0`. *Pass condition:* `GetX() == 0 && GetY() == 0 && getSpeed() == 0`.

TC-C02 — Acceleration Right

Setup: Place a `Buggy` at `(0, 300)`. Set `tireLevel = 1`. *Action:* Call `setAcceleration(2)`, then call `updateX(0.016f)` ten times. *Expected:* `getSpeed()` is positive and `GetX()` is greater than 0. *Pass condition:* `getSpeed() > 0 && GetX() > 0`.

TC-C03 — Acceleration Left

Setup: Place a `Buggy` at `(500, 300)`. Set `tireLevel = 1`. *Action:* Call `setAcceleration(-2)`, then call `updateX(0.016f)` ten times. *Expected:* `getSpeed()` is negative and `GetX()` is less than 500. *Pass condition:* `getSpeed() < 0 && GetX() < 500`.

TC-C04 — Speed Cap

Setup: `Buggy` (`maxSpeed = 50`) at `(0, 300)`, `tireLevel = 1`. *Action:* Call `setAcceleration(2)` and run `updateX(0.016f)` 500 times. *Expected:* Speed is clamped and never exceeds 50 in either direction. *Pass condition:* `getSpeed() <= 50 && getSpeed() >= -50`.

TC-C05 — Friction Brings Car to Rest

Setup: Build up positive speed over 30 frames with `setAcceleration(2)`. *Action:* Call `setAcceleration(0)` and run `updateX(0.016f)` 300 more times. *Expected:* Speed decays to exactly zero under friction. *Pass condition:* `fabs(getSpeed()) < 0.001`.

TC-C06 — Jump Leaves Ground

Setup: `Buggy` at `(0, 300)`, grounded. *Action:* Call `jump()`, then `updateY(0.016f)` once. *Expected:* `GetY()` is less than 300. *Pass condition:* `GetY() < 300`.

TC-C07 — No Double Jump

Setup: `Buggy` at `(0, 300)`. *Action:* Call `jump()`, advance 5 frames via `updateY`, then call `jump()` again while still airborne. *Expected:* The second `jump()` call is ignored because `Grounded` is `false`. The car continues its original arc and does not gain extra upward velocity. *Pass condition:* Car is still airborne (`GetY() < 300`) and `upspeed` has not been reset to `jumpspeed`.

TC-C08 — Car Returns to Ground

Setup: `Buggy` at `(0, 300)`. *Action:* Call `jump()`, then run `updateY(0.016f)` 200 times. *Expected:* Gravity pulls the car back down and `GetY()` settles at exactly 300. *Pass condition:* `GetY() == 300.0`.

TC-C09 — Tyre Level Increases Traction

Setup: Two `Buggy` instances, both at `(0, 300)`. First with `tireLevel = 1`, second with `tireLevel = 5`. *Action:* Apply `setAcceleration(2)` to both and call `updateX(0.016f)` ten times on each. *Expected:* The level-5 car reaches a higher speed in the same number of frames. *Pass condition:* `speedL5 > speedL1`.

TC-C10 — Terrain Effect Reduces Speed

Setup: Build up speed over 30 frames with `setAcceleration(2)`. *Action:* Record `getSpeed()`, then call `applyTerrainEffect(0.4)` (Swamp modifier). *Expected:* Speed after the call is less than speed before. *Pass condition:* `speedAfter < speedBefore`.

TC-C11 — Terrain Speed Floor Prevents Zero Snap

Setup: `Buggy` with a tiny positive speed (less than 1.0) built up over a single small `dt`. *Action:* Call `applyTerrainEffect(0.4)`. *Expected:* Speed is clamped up to 1.0 before the modifier is applied, so the result is `1.0 * 0.4 = 0.4`, not zero. *Pass condition:* `getSpeed() >= 0` and car has not stopped dead.

7.2 Terrain Tests

TC-T01 — Speed Modifiers Are Correct

Setup: Construct one instance of each terrain type. *Action:* Call `getSpeedModifier()` on each. *Expected:* Grass = 1.0, Sand = 0.6, Swamp = 0.4, Ice = 1.4, Mud = 0.25, Rock = 0.8. *Pass condition:* All six return values match exactly.

TC-T02 — Symbols Are Correct

Setup: One instance of each terrain type. *Action:* Call `getSymbol()` on each. *Expected:* `Grass='.'`, `Sand=' '`, `Swamp='S'`, `Ice='I'`, `Mud='M'`, `Rock='R'`. *Pass condition:* All six match their defined character.

TC-T03 — Map Loops with Modulo

Setup: Call `TerrainManager::generateMap()`. *Action:* Call `getTerrain(0)` and `getTerrain(150)`. *Expected:* Both return a pointer to a tile with the same symbol, since index 150 wraps back to the start of the 150-tile original map. *Pass condition:* `getTerrain(0)->getSymbol() == getTerrain(150)->getSymbol()`.

TC-T04 — Large Index Does Not Crash

Setup: Initialised `TerrainManager`. *Action:* Call `getTerrain(999999)`. *Expected:* Returns a valid non-null pointer without crashing or throwing. *Pass condition:* Returned pointer is not `nullptr`.

8 Build & Dependencies

Dependency	Notes
C++ Standard	C++17 or later
Raylib	4.x
Compiler	GCC / Clang / MSVC

Linux:

```
g++ Main.cpp -o game -lraylib -lGL -lm -lpthread -ldl -lrt -lX11
    -std=c++17
```

Windows (MSYS2/MinGW):

```
g++ Main.cpp -o game.exe -lraylib -lopengl32 -lgdi32 -lwinmm
    -std=c++17
```

9 Revision History

Version	Date	Changes
0.1.0	March 2026	Initial prototype: car physics, terrain, parallax background
0.2.0	March 2026	Added zombies, gun, UI screens, new terrain types, new vehicles